

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Code and full notebook:

https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_6_RNN_LSTM_GRU

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

The following preprocessing steps were carried out:

1. Loaded the raw inertial signal files (body acceleration, body gyroscope and total acceleration, each with x/y/z components) from the UCI HAR dataset.
2. Arranged every observation window as 128×9 by stacking the nine signal channels.
3. Assigned one activity label to each window using the label files provided with the dataset.
4. Encoded the six activity labels (1–6) into class indices (0–5) using a label encoder.
5. Normalized every channel using the mean and standard deviation computed from the *training* split only, and applied the same statistics to the validation and test splits.
6. Created training, validation and test sets using a stratified 70/15/15 split.
7. Verified that the class distribution remained balanced after splitting.

Laboratory subset used: From the full pool of 10,299 windows, a balanced subset of 3000 windows was drawn (500 windows per activity, chosen uniformly at random) to keep training time manageable, as recommended in the experiment sheet.

Actual Preprocessing Output

Running the preprocessing pipeline on the selected subset produced:

Input tensor shape:

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (450, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

Class distribution (train): 350 windows per class (all 6 classes)

Class distribution (val) : 75 windows per class (all 6 classes)

Class distribution (test) : 75 windows per class (all 6 classes)

The stratified split kept the classes exactly balanced across all three partitions, which means the accuracy metric reported later is not biased by class imbalance. The test set was kept completely untouched until the final evaluation step.

6. Temporal Data Visualization

Plot 1: Sensor signal versus time

Figure 1 shows three representative channels (`body_acc_x`, `body_gyro_x` and `total_acc_x`) plotted against the time step (1–128) for one representative window from three different activities: WALKING, SITTING and LAYING.

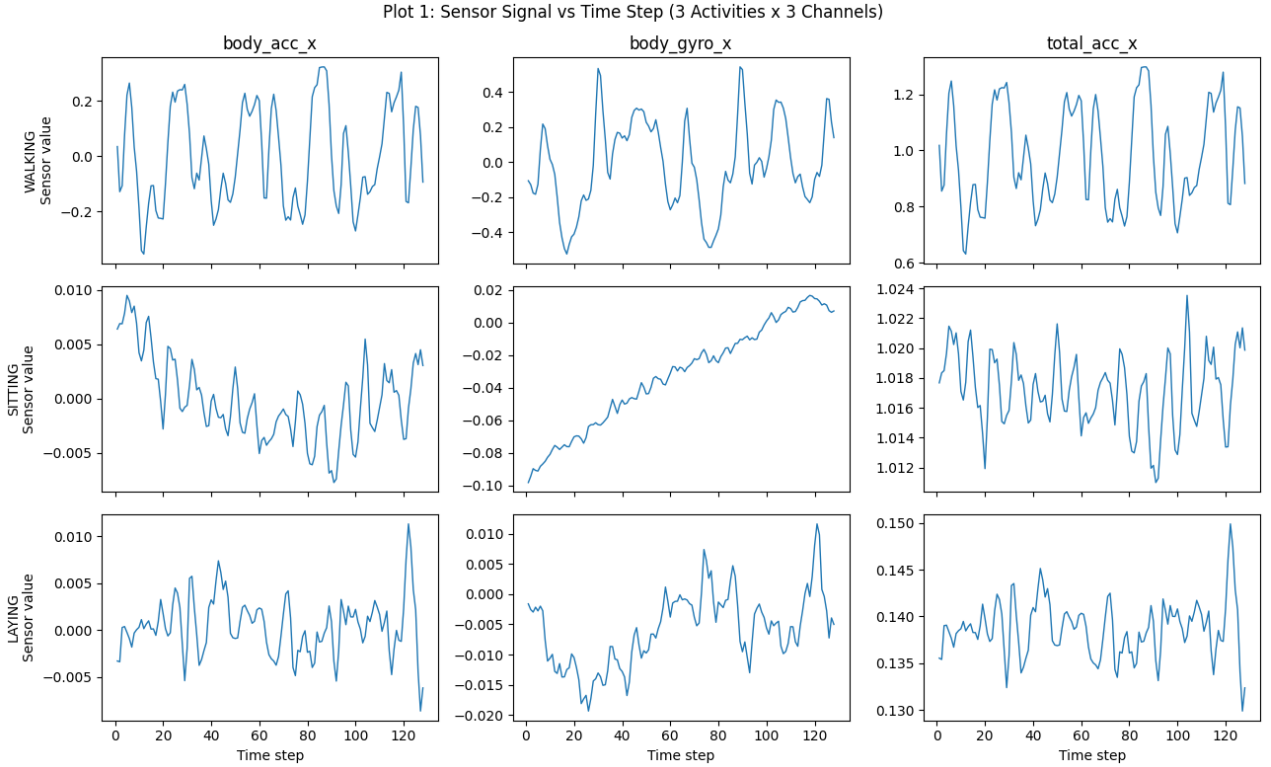


Figure 1: Plot 1: Sensor signal vs. time step for three activities (WALKING, SITTING, LAYING) across three channels.

Required inference:

1. **What temporal pattern is visible?** WALKING shows a clear, repeating oscillatory pattern in all three channels – this is the periodic swing of the leg/torso during each step, so the signal rises and falls with a fairly regular period across the 128 time steps. SITTING and LAYING, on the other hand, have no repeating pattern and a very small amplitude (about 0.01 for `body_acc_x` compared with about 0.3 for WALKING); they show only noise-like fluctuations (and a slow drift in the SITTING gyroscope channel), because the body is stationary and

the sensor only picks up small vibrations and the constant pull of gravity (which is why the `total_acc` channel sits at a different baseline level for each posture, about 1.02 for SITTING and 0.14 for LAYING).

2. **Which activities exhibit similar patterns?** SITTING and LAYING look the most similar to each other – both are static postures with low amplitude, noise-like signals and no periodicity. Their baselines are different, though, which is why LAYING is later classified perfectly. The confusion matrices show that the static pair SITTING and STANDING is the one most often mixed up by the models. The three "dynamic" activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS) also resemble each other in that all three are periodic, differing mainly in amplitude and exact shape rather than in whether a pattern exists at all.
3. **Why is the ordering of measurements important?** The activity is defined by *how the signal evolves over time*, not by the individual sample values. If the 128 time steps were shuffled, the periodic gait pattern of WALKING would be destroyed and would look like random noise, very similar to SITTING or STANDING. This is exactly why a recurrent model (which processes the sequence in order and carries information forward through its hidden state) is used instead of a simple feed-forward network that would treat every time step independently.

7. Vanilla RNN Architecture

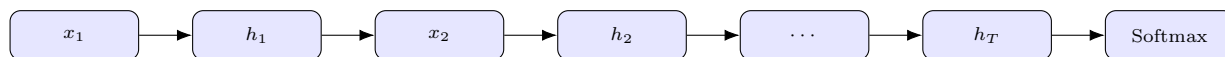
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

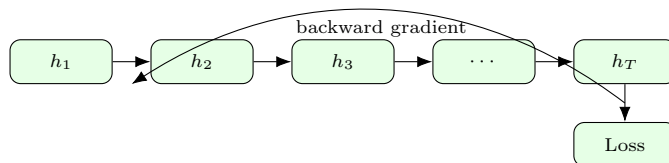
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$, the values were first worked out by hand and then verified with a short program.

Hand calculation:

$$h_1 = \tanh(0.5 \times 0.5 + 0.8 \times 0 + 0.1) = \tanh(0.35) = 0.3364$$

$$h_2 = \tanh(0.5 \times 0.7 + 0.8 \times 0.3364 + 0.1) = \tanh(0.7191) = 0.6164$$

$$h_3 = \tanh(0.5 \times 0.2 + 0.8 \times 0.6164 + 0.1) = \tanh(0.6931) = 0.6000$$

Program output:

h1 = 0.336376

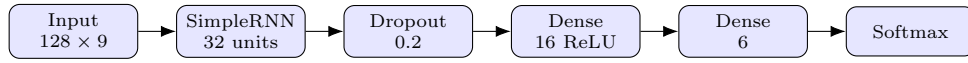
h2 = 0.616352

h3 = 0.599958

The hand-calculated values match the program output (small differences are only due to rounding during manual calculation), confirming that the update rule $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ was applied correctly.

9. RNN Implementation

The following recurrent architecture was used:



Training configuration used (Table 1):

Table 1: Training configuration used for the RNN, LSTM and GRU models.

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell C_t and three gates that control how information flows into, out of, and through that cell.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

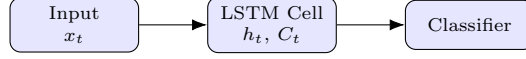
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is updated as

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: role of the cell state and gates. The cell state C_t behaves like a conveyor belt that runs across all time steps, and the gates only *add to* or *remove from* it through simple additive updates rather than through repeated multiplicative squashing. The forget gate decides how much of the old memory to keep, the input gate decides how much new information to write in, and the output gate decides how much of the memory to expose as the hidden state at that step. Because the cell-state update is additive ($C_t = f_t C_{t-1} + i_t \tilde{C}_t$) instead of a repeated matrix multiplication followed by a $\tanh$, gradients can flow backward through many time steps with much less shrinkage than in a Vanilla RNN, which is why LSTM can retain information over longer sequences.

11. GRU Architecture

GRU simplifies the LSTM design by merging the cell state and hidden state into a single state vector and by using only two gates.

The update gate is

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z),$$

and the reset gate is

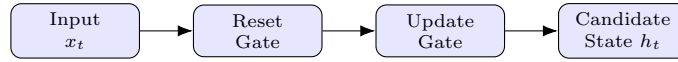
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



Comparison of gates and states: A Vanilla RNN has only a hidden state and no gates. LSTM has both a hidden state and a separate cell state, and uses three gates (forget, input, output). GRU keeps only a single hidden state and uses two gates (update, reset). Fewer gates means fewer weight matrices to learn, so GRU is a lighter-weight alternative to LSTM that, in this experiment, still performs at least as well while training a little faster (see Section 16).

12. LSTM and GRU Implementation

The same classifier head used for the RNN (Section 9) was reused for LSTM and GRU, with only the recurrent layer swapped out, so that the comparison between the three models is controlled (Table 2):

Table 2: Recurrent layer used in each of the three models (same classifier head for all).

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions (optimizer Adam with learning rate 10^{-3} , batch size 32, 30 epochs, sparse categorical cross-entropy loss, and the same train/validation/test split) were kept identical for all three models.

13. Required Training Plots

Plot 2: Training and Validation Loss Plot 3: Training and Validation Accuracy

Figure 2 shows the training/validation loss (top row) and training/validation accuracy (bottom row) for RNN, LSTM and GRU over the 30 training epochs.

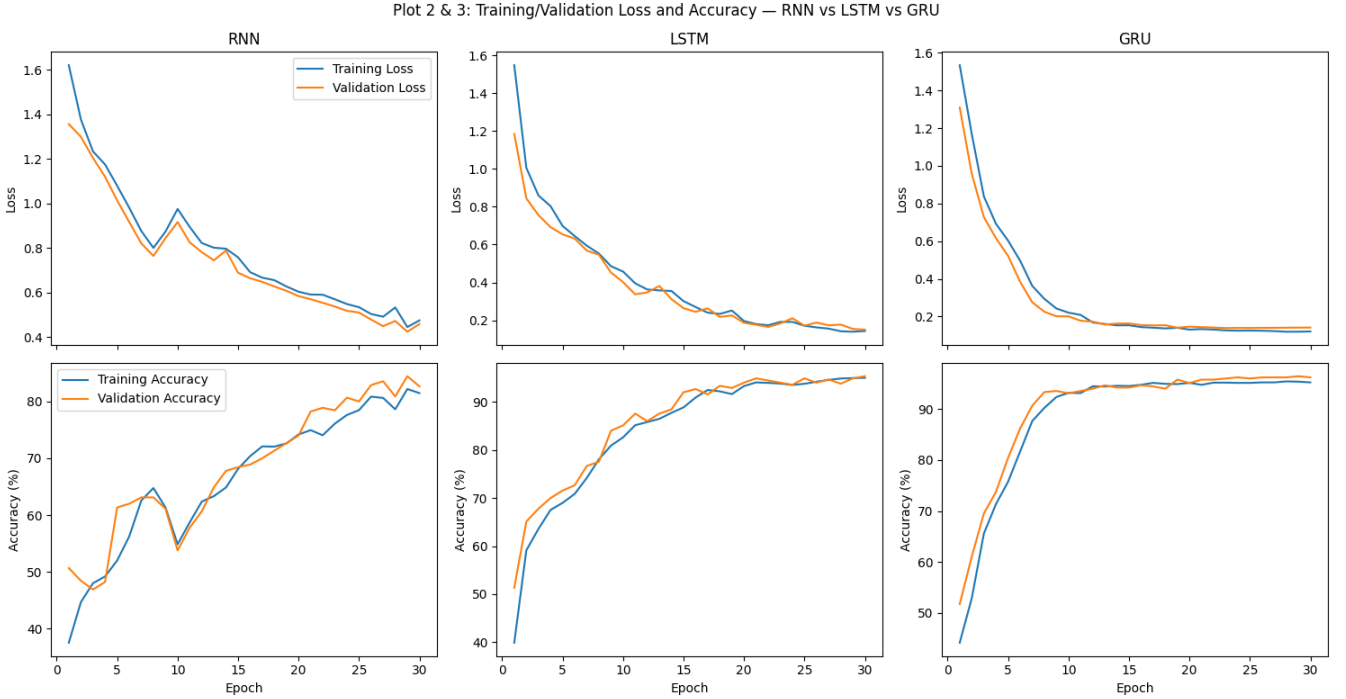


Figure 2: Plot 2 & 3: Training/validation loss and accuracy curves for RNN, LSTM and GRU.

Required inference:

- **RNN:** The RNN converges the slowest and the least smoothly of the three models. Both the training and validation loss decrease overall, but with visible bumps – for example, around epoch 8–10 the loss briefly rises again before continuing to fall, and a smaller bump appears near epoch 28. This instability is a direct symptom of the vanishing/exploding gradient issue discussed in Section 8: because SimpleRNN has to push gradient information through 128 raw multiplicative time steps, training is noisier. Even after 30 epochs the loss has not fully flattened, so the RNN would likely benefit from more epochs.
- **LSTM:** LSTM converges considerably faster and more smoothly than the RNN. Training and validation loss fall together for the first ~ 15 epochs, and by around epoch 20 both curves flatten out close to each other at a low loss value (≈ 0.15 – 0.2), with training and validation accuracy both around 94–95%. The two curves track each other closely throughout, so there is no visible overfitting.
- **GRU:** GRU shows the fastest and smoothest convergence of the three models. Loss drops sharply within the first 7–8 epochs and then plateaus near 0.12–0.15 for the remainder of training, with training and validation curves almost overlapping from epoch ~ 12 onward. Validation accuracy is consistently at or slightly above training accuracy, which is expected here because dropout (0.2) is active only during training and is switched off during validation, effectively making the validation-time network a little more powerful.
- **Overfitting / underfitting:** None of the three models show strong overfitting – the validation curves never diverge upward away from the training curves. The RNN instead shows mild *underfitting* relative to LSTM/GRU: after the same 30 epochs its training accuracy (≈ 81 – 82%) is still well below LSTM’s and GRU’s ($\approx 95\%$), i.e. the RNN simply has not learned the task as well in the same training budget, consistent with it struggling more with the long, 128-step sequences.

14. Performance Evaluation

Each model was evaluated once on the held-out, untouched test set (450 windows, 75 per activity). The mandatory metrics are macro-averaged since this is a balanced six-class classification problem. The results are given in Table 3.

Table 3: Test-set performance, parameter count and training time (450 test windows).

Metric	RNN	LSTM	GRU
Accuracy (%)	80.67	93.56	95.33
Macro Precision (%)	80.79	93.66	95.33
Macro Recall (%)	80.67	93.56	95.33
Macro F1 (%)	80.68	93.55	95.32
Parameters	1974	6006	4758
Training Time (s)	29.64	29.51	24.69

GRU achieved the highest test accuracy and macro F1-score, closely followed by LSTM, while the plain RNN trails both gated architectures by roughly 13 to 15 percentage points despite using the fewest parameters.

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Figure 3 shows the test-set confusion matrix for each model (rows = true label, columns = predicted label).

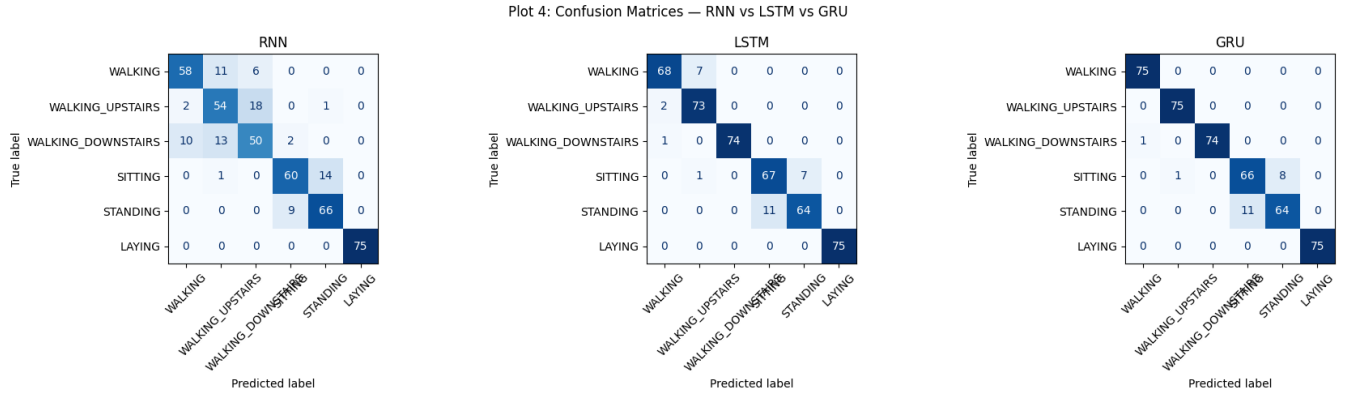


Figure 3: Plot 4: Confusion matrices for RNN, LSTM and GRU on the test set (75 windows per class).

Required inference:

- **Activity with the highest recognition rate:** LAYING is recognized perfectly by all three models (75/75 correct in every case). WALKING_UPSTAIRS is also recognized very well by LSTM and GRU (73/75 and 75/75 respectively).
- **Activity with the largest number of misclassifications:** For the RNN, WALKING_DOWNSTAIRS is the weakest class – only 50 of 75 windows are classified correctly, with 10 confused as WALKING and 13 as WALKING_UPSTAIRS. For LSTM and GRU, the weakest class is STANDING (64/75 correct for both), which is mostly confused with SITTING.
- **Pairs of activities frequently confused:** Two confusion pairs stand out across all three models: (i) the three walking-related activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS) are confused with each other, which is expected because they share very similar periodic gait signals and differ mainly in incline; and (ii) SITTING and STANDING are confused with each other in all three models (e.g. GRU confuses 8 SITTING windows as STANDING and 11 STANDING windows as SITTING), which matches the observation from Plot 1 that both are static postures with very similar, low-amplitude signals.
- **Consistency of errors across models:** The *pattern* of confusion is consistent across all three models – confusions happen almost exclusively within the "dynamic" group (WALKING/UPSTAIRS/DOWNSTAIRS) or within the "static" group (SITTING/STANDING), and only very rarely across the two groups (at most 1–3 windows per class, for example WALKING is never confused with SITTING). What changes across models is the *amount* of error: out of the 225 windows belonging to the three dynamic activities, the RNN misclassifies 63 of them, LSTM misclassifies only 10, and GRU misclassifies just 1, showing that the gated models are much better at telling apart the three walking activities, which differ mainly in fine temporal detail rather than gross shape.

16. RNN vs. LSTM vs. GRU Comparison

Table 4 compares the three architectures.

Table 4: Comparison of RNN, LSTM and GRU.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	29.6	29.5	24.7
Test F1-score (%)	80.68	93.55	95.32

Plot 5: Model Performance Comparison

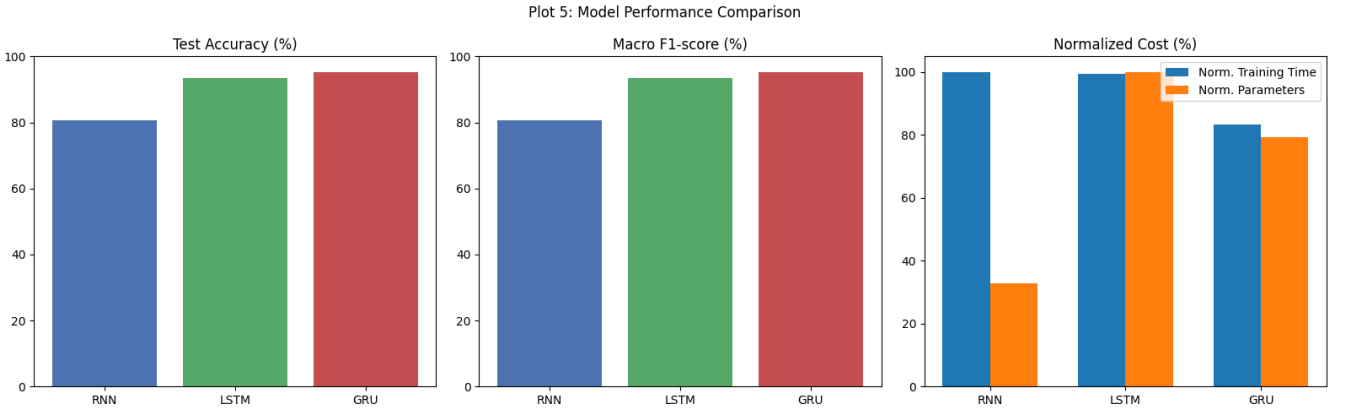


Figure 4: Plot 5: Test accuracy, macro F1-score, and normalized training time/parameter count for RNN, LSTM and GRU.

Required inference – trade-off among predictive performance, model complexity and training cost: GRU gives the best result on every accuracy metric while also having fewer parameters than LSTM (4758 vs. 6006) and the shortest training time of the three (24.7s vs. ≈ 29.5 s for RNN and LSTM). This shows that, at least for this task, GRU’s simpler two-gate design is not a disadvantage – it reaches a good bias/variance trade-off with less machinery than LSTM. The plain RNN is the cheapest model in terms of parameters (1974), and its per-epoch training time is similar to the others’, but this “savings” comes at a real cost in accuracy: it is about 13 to 15 points behind the gated models on every metric, which is the direct result of the gradient problems discussed in Section 8 – with 128 time steps to unroll, the RNN simply cannot learn the long-range temporal pattern as reliably as LSTM or GRU can. Overall, the results support the general rule of thumb that gated recurrent units are worth their extra parameters whenever sequences are long enough for vanishing gradients to matter, and that GRU is often a good default choice over LSTM when a slightly lighter, faster-training model is preferred.

17. Effect of Sequence Length

The experiment was repeated for $T \in \{32, 64, 128\}$ by keeping only the last T time steps of every 128×9 window (same normalization statistics, same train/validation/test split), and training a fresh RNN, LSTM and GRU model

Table 5 gives the test macro F1-score and training time for each T .

Table 5: Test macro F1-score (%) and training time (s) for different sequence lengths T .

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)	RNN Time (s)	LSTM Time (s)	GRU Time (s)
32	82.66	93.54	93.33	19.37	18.51	17.84
64	78.97	93.76	95.10	20.85	20.02	19.47
128	76.44	95.54	94.65	28.21	23.04	23.21

Note: The RNN F1-score at $T=128$ in Table 5 (76.44%) is lower than the 80.68% in Table 3, although the configuration is identical. This is a separate training run, so the random weight initialization and dropout are different. It shows that the RNN results vary from run to run, and each number here comes from a single run.

Plot 6: Sequence Length vs. Test F1-score

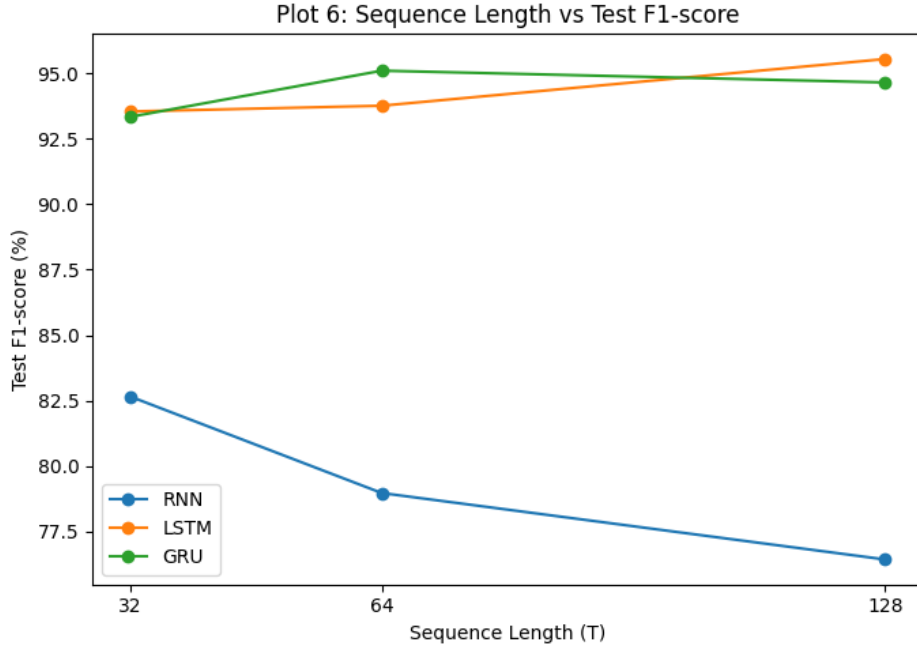
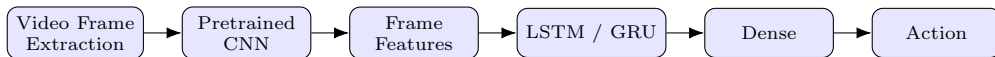


Figure 5: Plot 6: Test F1-score vs. sequence length T for RNN, LSTM and GRU.

Required inference: The RNN’s F1-score gets worse as the sequence grows longer (82.66% at $T=32$ down to 76.44% at $T=128$). This is the clearest evidence in this experiment of the vanishing-gradient problem from Section 8 – the longer the sequence the RNN has to unroll over, the harder it is for gradient information from the loss to reach the early time steps, so a plain RNN actually does relatively better on shorter sequences that it can fully remember. LSTM and GRU behave very differently: both stay roughly flat or even improve slightly as T increases (LSTM: 93.54% to 95.54%; GRU: 93.33% to 95.10% at $T=64$ before settling at 94.65% at $T=128$), confirming that their gating mechanisms let them make use of the extra temporal context in longer sequences instead of being hurt by it. The overall conclusion is that sequence length interacts strongly with architecture choice: more temporal context only helps if the model is actually able to propagate gradients across that context, which is exactly what LSTM’s and GRU’s gates are designed to do. Training time also increased with sequence length: from 19.4/18.5/17.8 s at $T=32$ to 28.2/23.0/23.2 s at $T=128$ for RNN/LSTM/GRU, showing the additional computational cost of longer sequences.

A video is a sequence of frames, so the same idea used for the sensor data can be reused here: a pretrained CNN first extracts a spatial feature vector from each frame, and a recurrent network (LSTM or GRU) then models how those per-frame features change over time.



Dataset used: A small subset of the UCF101 action-recognition dataset was used, restricted to 3 action classes – IceDancing, WalkingWithDog and YoYo – with a small number of videos per class. uploaded to Google Drive and

Required inference: The RNN’s F1-score gets worse as the sequence grows longer (82.66% at $T=32$ down to 76.44% at $T=128$). could not be reliably extracted, 43 usable videos remained.

19. Expected Video Input and Output

10 frames were sampled uniformly from each video and each frame was resized to $224 \times 224 \times 3$, giving a raw frame tensor of shape

$$(43, 10, 224, 224, 3).$$

After CNN feature extraction (Section 20), each video is represented as a sequence of 10 feature vectors, which is what is actually fed to the recurrent network:

$$X \in \mathbb{R}^{43 \times 10 \times D}.$$

The output of the trained CNN-LSTM/CNN-GRU model is a probability vector over the 3 action classes (softmax), and the predicted class is the class with the highest probability. In the notebook the predictions of the 7 test videos were compared with the true labels through the confusion matrix (Plot 9). The updated run also reports the per-video softmax confidence; the table below lists the predicted class, actual class and confidence for each test video.

20. CNN Feature Extraction

A pretrained MobileNetV2 (ImageNet weights, classification head removed, global average pooling) was used strictly as a frozen feature extractor – it was not trained or fine-tuned in any way.

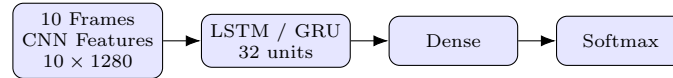


Required observation: MobileNetV2 produced a feature vector of dimension $D = 1280$ per frame. Stacking the 10 sampled frames per video, the resulting tensor supplied to the recurrent network has shape

$$(43, 10, 1280).$$

21. CNN-LSTM / CNN-GRU Model

The same classifier-head builder function used for the sensor-data models (Section 9/12) was reused here, with sequence length 10 and feature dimension 1280 in place of sequence length 128 and 9 channels:



The 43 videos were split 70/15/15 (stratified by class) into 30 training, 6 validation and 7 test videos.

Plot 7: Video Sample Frames

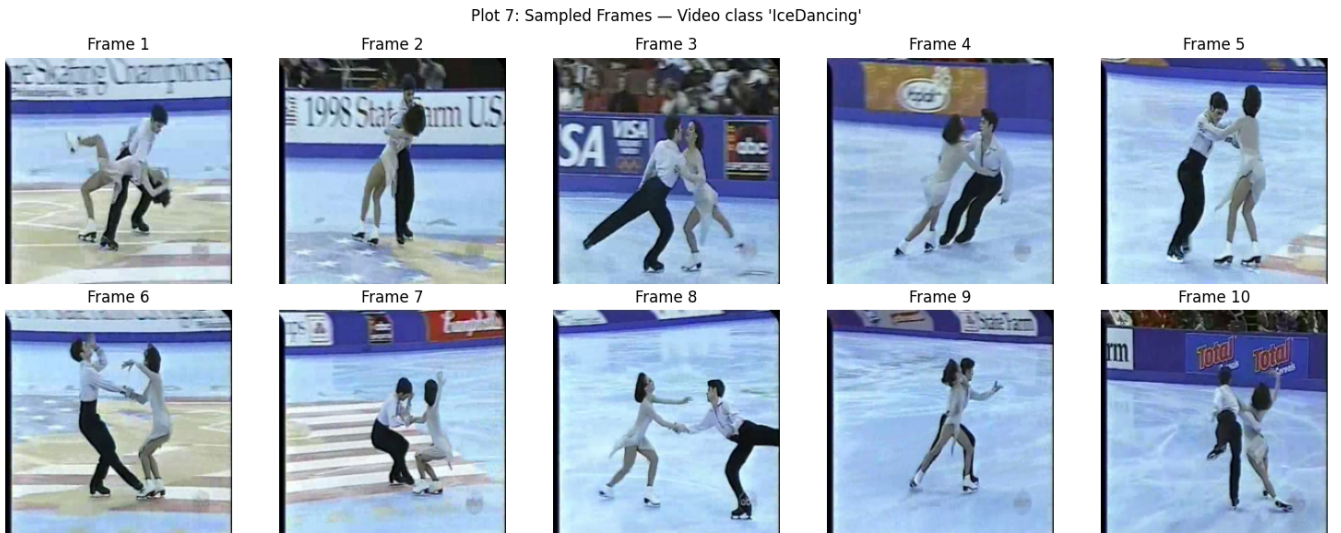


Figure 6: Plot 7: 10 uniformly sampled frames from one representative video.

Plot 8: Video Training and Validation Curves

Per-video confidence (7 held-out test videos)

CNN-LSTM			
Video	Pred.	Actual	Conf.
1	IceDancing	IceDancing	0.9915
2	YoYo	YoYo	0.9843
3	WalkingWithDog	WalkingWithDog	0.9780
4	IceDancing	IceDancing	0.9900
5	WalkingWithDog	WalkingWithDog	0.9828
6	YoYo	YoYo	0.9845
7	IceDancing	IceDancing	0.9906

CNN-GRU			
Video	Pred.	Actual	Conf.
1	IceDancing	IceDancing	0.8911
2	YoYo	YoYo	0.9779
3	WalkingWithDog	WalkingWithDog	0.9382
4	IceDancing	IceDancing	0.8970
5	WalkingWithDog	WalkingWithDog	0.9195
6	YoYo	YoYo	0.9823
7	IceDancing	IceDancing	0.8830

Plot 8: Video Task — Training/Validation Curves

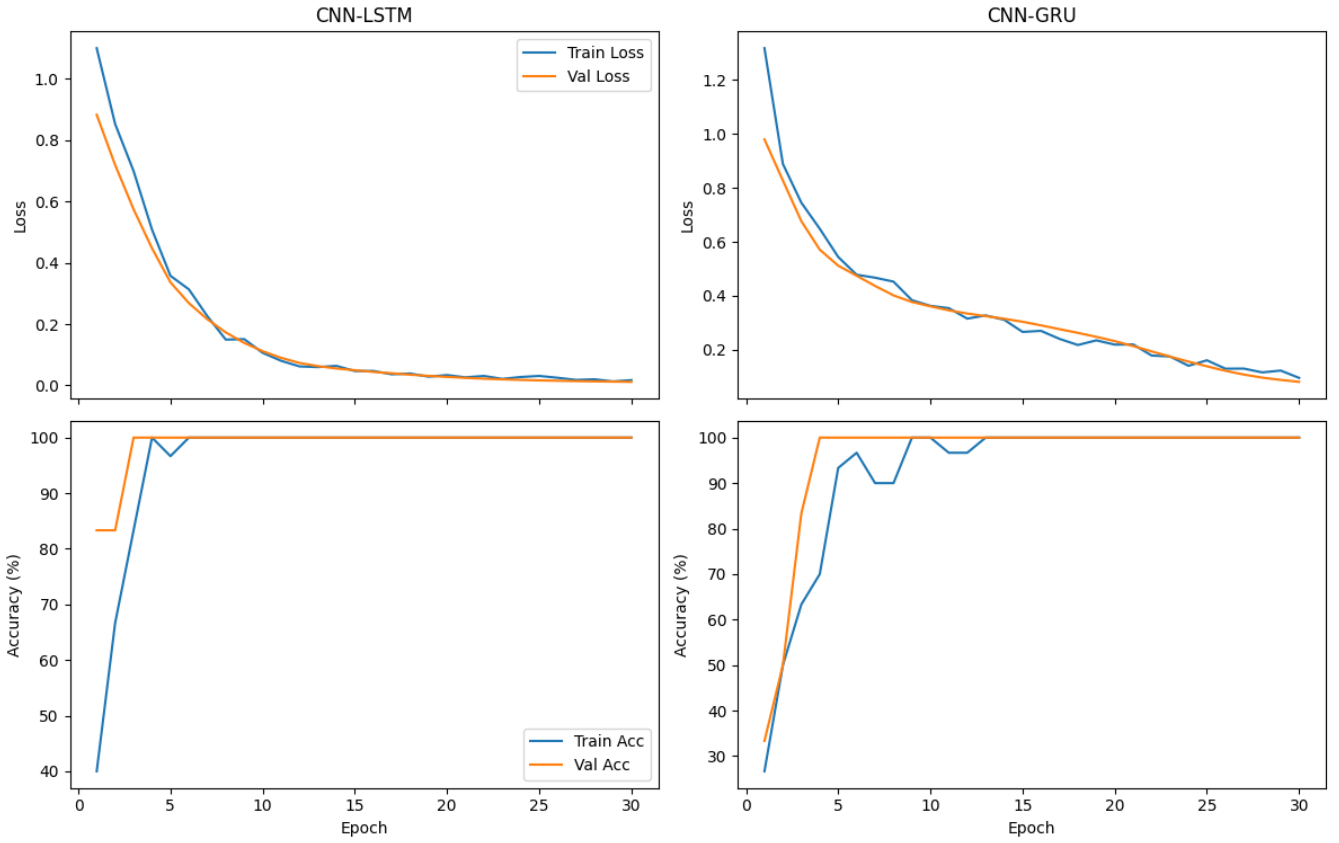


Figure 7: Plot 8: Training/validation loss and accuracy for CNN-LSTM and CNN-GRU.

Plot 9: Video Confusion Matrix

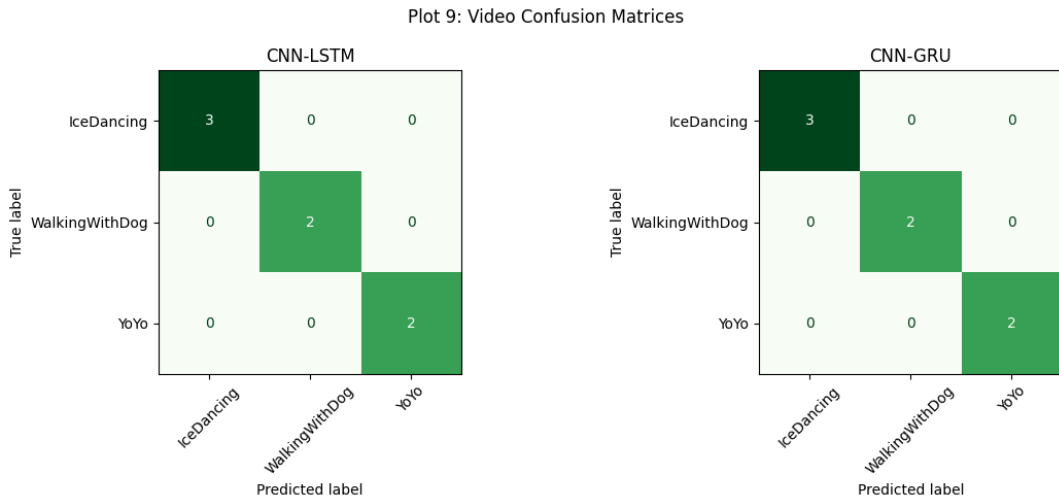


Figure 8: Plot 9: Test-set confusion matrices for CNN-LSTM and CNN-GRU (7 test videos: 3 IceDancing, 2 WalkingWithDog, 2 YoYo).

Inference from Plot 8 and Plot 9: Plot 8 shows the training and validation loss and accuracy of the two video models over 30 epochs. Both losses fall steadily and both validation accuracies reach 100% within the first few epochs (epoch 3 for CNN-LSTM and epoch 4 for CNN-GRU). CNN-LSTM's loss falls faster (final training/validation loss 0.0174/0.0117) than CNN-GRU's (0.0959/0.0813), although both have the same accuracy in the end. This happens because the frozen MobileNetV2 features are already very informative for these three visually different actions, so the recurrent layer only has to learn a simple decision on top of them. The validation set has only 6 videos, so these curves are not very reliable. Plot 9 shows no class-level or temporal confusion at all: all 3 IceDancing, 2 WalkingWithDog and 2 YoYo test videos are classified correctly by both models.

Test performance: Both CNN-LSTM and CNN-GRU reached 100% test accuracy, precision, recall and F1-score

on the 7 held-out test videos (Plot 9 shows every video correctly classified). CNN-LSTM used 168,643 trainable parameters and CNN-GRU used 126,723.

Required inference: The CNN (MobileNetV2) captures *spatial* information from each individual frame – the objects, textures, poses and scene layout visible in that single instant, e.g. a person’s body pose, the presence of a dog, or a yo-yo and hand position – without any notion of what came before or after. Because the CNN is frozen and only pooled globally, its 1280-d output per frame is essentially a compact “what is in this picture” descriptor. The recurrent network (LSTM/GRU) then learns the *temporal* information – how those per-frame descriptors evolve across the 10 sampled frames, which is what actually distinguishes an action from a static pose (e.g. the repeating arm swing of walking a dog vs. the spinning motion of a yo-yo vs. the turning motion of ice dancing). Both recurrent variants reach 100% test accuracy here mainly because the 3 chosen action classes are visually very different from one another, so even individual CNN frame features are already close to linearly separable; with only 7 test videos, this also means the perfect score should be read as “no errors on a very small test set” rather than as proof that the video pipeline is flawless – a larger, more fine-grained action set (e.g. distinguishing different sports) would be a harder and more realistic test of the temporal modeling.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another. As the laboratory task, a synthetic *reversal* dataset was generated, for example $[1, 4, 7, 2] \rightarrow [2, 7, 4, 1]$. In total 8000 random sequences of length 4 were created using the integers 1 to 9. The data was split into 6800 training and 1200 test sequences (15% test), and 10% of the training sequences (680) were used for validation during training.



The encoder LSTM reads the input sequence and its final hidden and cell states form the context. The decoder LSTM is initialized with this context and generates the output one element at a time. Both the encoder and the decoder use an embedding layer, and the decoder ends with a softmax layer over the vocabulary. Table 6 lists the configuration used.

Table 6: Encoder–decoder configuration for the reversal task.

Parameter	Value
Vocabulary size	11 (digits 1–9; index 0 is used as the start token)
Embedding dimension	32
Encoder / decoder	LSTM, 64 units each
Output layer	Dense (11), softmax
Optimizer / loss	Adam / sparse categorical cross-entropy
Batch size / epochs	64 / 30
Trainable parameters	51,083

Teacher forcing: During training the decoder input is the start token followed by the *correct* target sequence shifted right by one position, so at every step the decoder sees the true previous token instead of its own earlier prediction. At test time this is not possible, so the output was generated by *greedy decoding*: the decoder starts with the start token, picks the most probable token, and feeds it back as the next input.

23. Expected Sequence-to-Sequence Input and Output

Table 7 shows five examples from the held-out test set. The predicted sequences were generated by the trained model with greedy decoding. In all five cases the predicted output is exactly the reverse of the input sequence.

Table 7: Five test examples of the sequence reversal task.

Sample	Input Sequence	Predicted Output
1	[2, 2, 4, 6]	[6, 4, 2, 2]
2	[8, 6, 3, 4]	[4, 3, 6, 8]
3	[6, 3, 7, 7]	[7, 7, 3, 6]
4	[1, 5, 1, 6]	[6, 1, 5, 1]
5	[7, 2, 9, 8]	[8, 9, 2, 7]

24. Sequence-to-Sequence Evaluation

The trained model was evaluated on all 1200 test sequences (4800 tokens) using greedy decoding. Token accuracy is the number of correctly predicted tokens divided by the total number of tokens, and sequence accuracy is the number of completely correct sequences divided by the total number of sequences.

Table 8: Sequence-to-sequence evaluation results.

Metric	Value
Token accuracy	100.00%
Sequence accuracy	100.00%
Final training loss (epoch 30)	0.0008
Final validation loss (epoch 30)	0.0009

During training, the validation accuracy (teacher-forced) was 42.13% after epoch 1, 94.38% after epoch 2 and 99.85% after epoch 3, and it reached 100% by epoch 6. The training loss went from 1.9744 in epoch 1 to 0.0008 in epoch 30, and the validation loss stayed very close to it (0.0009 at the end), so there is no sign of overfitting.

Required inference: Token accuracy counts every position separately, but a sequence is counted as correct only if *all* its tokens are correct. So a single wrong token makes the whole sequence wrong, and sequence accuracy is always less than or equal to token accuracy. For example, if each of the 4 tokens were predicted correctly with probability 0.95 independently, the token accuracy would be 95% but the sequence accuracy would only be about $0.95^4 \approx 81\%$. The gap gets larger for longer sequences, and in a decoder it can get even bigger because an early mistake is fed back as input and can cause more mistakes later. In this experiment the model made no mistakes on the 1200 test sequences, so both accuracies are 100% and this gap could not be observed. The example above is only an illustration of the reasoning and not a result of the experiment. The reversal task with 4 short sequences is very easy for an LSTM encoder-decoder, which is why it is solved perfectly.

25. Consolidated Results

Table 9 collects the test results of all classification models (all values are from the notebook). The RNN, LSTM and GRU rows are for the 450 HAR test windows and the CNN-LSTM/GRU rows are for the 7 test videos. The metrics are macro-averaged.

Table 9: Consolidated test results of all classification models.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	80.67	80.79	80.67	80.68	1974
LSTM	93.56	93.66	93.56	93.55	6006
GRU	95.33	95.33	95.33	95.32	4758
CNN-LSTM	100.00	100.00	100.00	100.00	168,643
CNN-GRU	100.00	100.00	100.00	100.00	126,723

The parameters of the CNN-LSTM/GRU models include only the recurrent classifier, because the MobileNetV2 feature extractor was frozen and used only to compute features. The video results are based on just 7 test videos, so they should not be compared directly with the HAR results.

The separate table for the sequence-to-sequence task is given below.

Table 10: Consolidated results of the sequence-to-sequence (reversal) task.

Model	Token Acc. (%)	Seq. Acc. (%)	Train Loss	Val. Loss	Parameters
Encoder-Decoder LSTM	100.00	100.00	0.0008	0.0009	51,083

26. Required Inferences

For every major plot, the inference is summarized below in the four-part format of the manual: what the plot shows, the trend, the reason, and the conclusion. The detailed inferences are in the sections mentioned.

1. **Temporal pattern in the sensor signals (Plot 1, Section 6).** *Shows:* three channels of WALKING, SITTING and LAYING windows. *Trend:* WALKING is clearly periodic with large amplitude, while SITTING

- and LAYING are low-amplitude and non-periodic. *Why:* walking moves the body in a repeating gait cycle, whereas static postures only produce small vibrations and a constant gravity component. *Conclusion:* the activity is determined by how the signal changes over time, so the order of the samples matters and a sequence model is appropriate.
2. **Convergence of the RNN (Plot 2/3, Section 13).** *Shows:* loss and accuracy over 30 epochs. *Trend:* the loss falls from 1.62 to 0.47 but slowly, with a visible bump around epochs 8–10, and the training accuracy ends near 81%. *Why:* vanishing/exploding gradients over 128 time steps make the training of a plain RNN unstable and slow. *Conclusion:* the RNN converges the slowest and was still improving at epoch 30.
 3. **Convergence of the LSTM (Plot 2/3, Section 13).** *Shows:* the same curves for the LSTM. *Trend:* smooth decrease of the loss (1.55 to 0.14), with the accuracy flattening around 94–95% after about 20 epochs. *Why:* the additive cell-state update and the gates let the gradient reach earlier time steps. *Conclusion:* the LSTM converges stably and reaches good accuracy within the 30 epochs.
 4. **Convergence of the GRU (Plot 2/3, Section 13).** *Shows:* the same curves for the GRU. *Trend:* the loss drops the fastest and is already close to its final value (about 0.12–0.14) by epoch 10–12, with the training and validation curves almost overlapping. *Why:* the GRU has a simpler gating structure with fewer parameters to learn than the LSTM, and it still keeps a gated path for the gradient. *Conclusion:* the GRU converged the fastest here.
 5. **Overfitting or underfitting (Plot 2/3, Section 13).** *Shows:* the gap between training and validation curves. *Trend:* validation loss stays close to the training loss (and validation accuracy is equal to or a little above the training accuracy) for all three models, so the generalization gap is small. *Why:* dropout is active only during training and the models are small (1974–6006 parameters). *Conclusion:* no strong overfitting; the RNN shows mild underfitting because its accuracy ($\approx 81\%$) is well below that of the LSTM and GRU ($\approx 95\%$).
 6. **Activity-wise errors (Plot 4, Section 15).** *Shows:* test confusion matrices. *Trend:* LAYING is perfectly recognized by all models; errors are mainly among WALKING, WALKING_UPSTAIRS and WALKING_DOWNSTAIRS (RNN: 63 of 225 wrong; LSTM: 10; GRU: 1) and between SITTING and STANDING (STANDING has 64/75 correct for LSTM and GRU). *Why:* the confused activities have similar signals (periodic gait differing only in incline; static postures with low amplitude). *Conclusion:* gated models mainly improve the separation of the three walking activities, while SITTING/STANDING remains the hardest pair.
 7. **Effect of sequence length (Plot 6, Section 17).** *Shows:* test F1 for $T = 32, 64, 128$. *Trend:* the RNN's F1 decreases (82.66% to 76.44%) as T increases, whereas LSTM (93.54% to 95.54%) and GRU (93.33% to 94.65%, best 95.10% at $T=64$) stay high or improve slightly. *Why:* longer sequences make the vanishing-gradient problem worse for the RNN, while the gates of LSTM/GRU can use the extra context. *Conclusion:* more temporal context helps only if the model can carry information across it. (These are single runs, so small differences should not be over-interpreted.)
 8. **Parameter and computational differences (Table 4, Plot 5, Section 16).** *Shows:* parameters, training time and F1 of each model. *Trend:* RNN 1974 parameters, GRU 4758 and LSTM 6006; training times are 29.6 s, 24.7 s and 29.5 s respectively (RNN and LSTM are almost the same); F1 is 80.68%, 95.32% and 93.55%. *Why:* an LSTM has four sets of weights per unit, a GRU has three and an RNN has one. *Conclusion:* the GRU gave the best accuracy with fewer parameters than the LSTM and the shortest training time, while the RNN is the smallest but the least accurate. Training times come from a single run each and depend on the hardware, so they are only indicative.
 9. **Role of the CNN in video understanding (Plot 7, Section 21).** *Shows:* the 10 sampled frames of a video. *Trend:* each frame is turned into a 1280-dimensional vector by the frozen MobileNetV2, giving a 10×1280 sequence per video. *Why:* the CNN, pretrained on ImageNet, describes the spatial content of a single frame (objects, pose, scene). *Conclusion:* the CNN acts as a spatial feature extractor, so the recurrent model does not have to learn from raw pixels with only 43 videos.
 10. **Role of the recurrent model in temporal modeling (Plot 8 and 9, Section 21).** *Shows:* training curves and confusion matrices of CNN–LSTM and CNN–GRU. *Trend:* both reach 100% validation and test accuracy. *Why:* the LSTM/GRU reads the 10 frame features in order and learns how they change over time (motion), which the CNN alone cannot see. *Conclusion:* the recurrent layer models the temporal side of the problem; however, with three very different classes and 7 test videos, the perfect score does not prove that temporal modeling was really needed.
 11. **Token accuracy vs. sequence accuracy (Section 24).** *Shows:* the evaluation of the reversal model. *Trend:* token and sequence accuracy are both 100% here. *Why:* sequence accuracy requires all tokens of a sequence to be correct, so in general it is lower than or equal to token accuracy. *Conclusion:* sequence accuracy is the stricter metric, and it should be reported together with token accuracy.

27. Discussion Questions

1. **What is a recurrent neural network?** An RNN is a neural network for sequential data. It processes the sequence one step at a time and keeps a hidden state that is updated at every step using the current input and the previous hidden state, with the same weights at all steps.

2. **What information is represented by the hidden state?** The hidden state is a compact summary of everything the network has seen in the sequence so far. It works as the memory of the network and is used for predictions.
3. **Why is sequence order important?** The meaning of the data depends on the order of the elements. In our HAR data the periodic pattern of walking would be destroyed if the 128 time steps were shuffled, and in the reversal task the same digits in a different order give a different output.
4. **Explain the difference between a feed-forward network and an RNN.** A feed-forward network maps a fixed-size input to an output with no memory and treats every input independently. An RNN has a recurrent connection, so its output at a step depends on the current input and on the previous states, and it can handle inputs of variable length.
5. **Explain Backpropagation Through Time.** BPTT unrolls the RNN over all time steps, treating each step as a layer that shares the same weights, and then applies normal backpropagation. The gradients from every step are propagated backward through the unrolled network and summed to update the shared weights.
6. **Why can gradients vanish during BPTT?** The gradient passes through the recurrent weights and the tanh derivative at each step. If these factors are smaller than 1, their product over many steps shrinks towards zero, so early time steps receive almost no learning signal.
7. **Why can gradients explode during BPTT?** If the repeated factors (mainly the recurrent weight matrix) are larger than 1, the product grows exponentially with the number of steps, so the gradients become very large and training becomes unstable. Gradient clipping is a common remedy.
8. **What are long-term dependencies?** They are relationships between elements that are far apart in a sequence, such that an output at a late step depends on an input from many steps earlier. Plain RNNs find them difficult to learn because of vanishing gradients.
9. **What is the role of the LSTM cell state?** The cell state C_t is the long-term memory of the LSTM. It is updated only by additive operations controlled by the gates, so information and gradients can flow across many time steps with little change.
10. **What are the forget, input and output gates in LSTM?** The forget gate decides how much of the previous cell state to keep, the input gate decides how much of the new candidate information to write into the cell state, and the output gate decides how much of the cell state is exposed as the hidden state.
11. **What are the reset and update gates in GRU?** The reset gate decides how much of the previous hidden state is used to compute the candidate state. The update gate decides how much of the old hidden state is kept and how much is replaced by the new candidate state.
12. **Compare LSTM and GRU.** The LSTM has a separate cell state and three gates, so it has more parameters (6006 vs. 4758 in our models). The GRU has one state and two gates, so it is simpler and usually a little faster to train. In our experiment the GRU had the higher test F1 (95.32% vs. 93.55%) and the shorter training time, but this is one dataset and one run, so it does not mean that the GRU is always better.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?** The GRU was designed to keep the ability of the gates to control memory while reducing the complexity. It merges the cell and hidden states and combines the forget and input gates into one update gate, which removes one gate and one state and reduces the number of weights.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?** If the data split, preprocessing, units, optimizer, learning rate, batch size and epochs differ, we cannot say whether a difference in results comes from the architecture or from the settings. Using identical settings makes the comparison fair.
15. **What does the confusion matrix reveal that accuracy alone does not?** Accuracy is only one number. The confusion matrix shows which classes are recognized well, which classes are confused with each other, and what kind of errors are made. For example, it showed that the LSTM and GRU errors were mainly between SITTING and STANDING.
16. **Why should macro F1-score be reported for multi-class classification?** Macro F1 computes the F1-score of each class and averages them with equal weight, so poor performance on a small class is not hidden by the large classes. It balances precision and recall and is more reliable than accuracy when classes are imbalanced (our test set was balanced, so macro F1 and accuracy were very close).
17. **What is the significance of sequence length?** It decides how much temporal context the model sees, and it also affects the computation and the difficulty of training. Longer sequences give more information but make gradient propagation harder. In Table 5 the RNN got worse as T increased, while LSTM and GRU did not.
18. **Why can a CNN be used as a feature extractor for video?** A CNN pretrained on a large image dataset such as ImageNet has learned general visual features (edges, textures, object parts). These features transfer to video frames, so the CNN can convert each frame into a compact feature vector without any retraining.
19. **What type of information is extracted by the CNN?** Spatial information from one frame at a time: the objects, textures, body pose and scene layout. It contains no information about motion between frames.
20. **What type of information is learned by the recurrent network?** Temporal information: how the frame features change from frame to frame, that is, the motion and the order of events, which is needed to recognize an action.

21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?** A video is an ordered series of frames, and an action is defined by the change over time. Arranging the frame features in time order as a $10 \times D$ sequence lets the recurrent network process them step by step and model that change.
22. **Explain the encoder and decoder in sequence-to-sequence learning.** The encoder reads the input sequence and compresses it into a context (the final hidden and cell states). The decoder is initialized with this context and generates the output sequence one element at a time, using the previously generated element as input.
23. **What is teacher forcing?** It is a training technique where the decoder is given the true previous target token as input at each step instead of its own prediction. It makes training faster and more stable, but at test time the decoder has to use its own predictions (as in our greedy decoding).
24. **Differentiate token accuracy and sequence accuracy.** Token accuracy is the fraction of individual output tokens that are correct. Sequence accuracy is the fraction of complete output sequences that are entirely correct. Sequence accuracy is stricter and never higher than token accuracy.
25. **Why must the test set remain untouched during model selection?** If the test set is used to choose models or tune hyperparameters, information from it leaks into the choices, and the final test score becomes an optimistic estimate. Keeping it untouched gives an unbiased estimate of performance on new data. In our experiment the validation set was used during training and the test set was used only for the final evaluation.

28. Additional Exercises

The additional exercises were completed where new notebook runs were required. Table 11 summarizes the status; detailed results are given on the following page.

Table 11: Status of the additional exercises.

No.	Exercise	Status
1	16 and 64 recurrent units	Completed; 16/32/64 units tested for all three models. See Table 12.
2	GRU vs. LSTM with the same units	Covered by the main comparison; both use 32 units (Table 3); GRU F1 95.32% vs. LSTM 93.55%.
3	Second recurrent layer	Completed; 2-layer RNN/LSTM/GRU compared with 1-layer baselines. See Table 13.
4	Bidirectional vs. unidirectional LSTM	Completed; BiLSTM compared with the 32-unit unidirectional LSTM. See Table 14.
5	Sequence length and computational cost	Completed; F1 and training time recorded for T=32, 64, 128 (Table 5).
6	LSTM vs. GRU on identical CNN features	Covered: both use identical CNN features and reach 100% test accuracy on 7 videos.
7	Seq2seq with different output length	Completed; input length 5 - output length 3. Token and sequence accuracy = 100%. See Table 15.

29. Expected Outcome

In this experiment the complete sequence-learning pipeline was implemented:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

as well as the encoder-decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

On the HAR data, the gated models clearly performed better than the plain RNN (test F1: RNN 80.68%, LSTM 93.55%, GRU 95.32%), and the gap grew with longer sequences, which agrees with the vanishing-gradient explanation. The GRU gave the best result with fewer parameters and less training time than the LSTM. The CNN-LSTM/GRU pipeline classified all 7 test videos of the three UCF101 classes correctly, and the encoder-decoder LSTM reversed all 1200 test sequences correctly. The small size of the video test set and the simplicity of the reversal task should be kept in mind when interpreting these results, and all results come from single training runs.

28. Additional Exercise Results

The following results are from the newly executed notebook cells. The existing 32-unit one-layer results already reported in the main experiment are retained unchanged as the baselines.

Additional Exercise 1: Effect of recurrent units (16 vs. 32 vs. 64)

Table 12: Accuracy, macro F1-score, parameter count and training time.

Model	Units	Accuracy (%)	F1 (%)	Parameters	Time (s)
RNN	16	75.78	75.62	790	29.75
RNN	32*	80.67	80.68	1,974	29.64
RNN	64	81.56	81.55	5,878	29.69
LSTM	16	86.44	86.43	2,038	29.49
LSTM	32*	93.56	93.55	6,006	29.51
LSTM	64	95.33	95.32	20,086	25.10
GRU	16	94.89	94.87	1,670	23.80
GRU	32*	95.33	95.32	4,758	24.69
GRU	64	95.11	95.10	15,542	26.63

* The 32-unit baseline values are the existing main-experiment results retained from the original report; the 16- and 64-unit rows are from the updated notebook. Each setting is a single run.

Additional Exercise 3: Second recurrent layer (stacked)

Table 13: One-layer baseline versus two-layer recurrent models.

Model	Layers	Accuracy (%)	F1 (%)	Parameters	Time (s)
RNN	1*	80.67	80.68	1,974	29.64
RNN	2	67.33	65.76	4,054	47.41
LSTM	1*	93.56	93.55	6,006	29.51
LSTM	2	94.89	94.86	14,326	35.96
GRU	1*	95.33	95.32	4,758	24.69
GRU	2	95.56	95.57	11,094	36.31

* One-layer values are the existing main-experiment baselines. The added recurrent layer increased parameter count and training time in all three cases; LSTM improved slightly, while RNN decreased and GRU changed very little.

Additional Exercise 4: Bidirectional vs. unidirectional LSTM

Table 14: LSTM comparison.

Model	Accuracy (%)	F1 (%)	Parameters	Time (s)
LSTM (unidirectional)*	93.56	93.55	6,006	29.51
Bidirectional LSTM	95.56	95.55	11,894	33.53

* Existing 32-unit unidirectional LSTM baseline. The bidirectional model improved F1 by about 2.0 percentage points relative to that baseline, at the cost of more parameters and training time.

Additional Exercise 7: Seq2seq with different output length

The modified task used input length 5 and output length 3; the target was the reversed first three input elements. Test token accuracy and sequence accuracy were both 100.00%.

Table 15: Five examples from the different-length seq2seq task.

Sample	Input Sequence	Predicted Output	Ground Truth
1	[8, 9, 7, 7, 2]	[7, 9, 8]	[7, 9, 8]
2	[8, 2, 7, 2, 1]	[7, 2, 8]	[7, 2, 8]
3	[9, 5, 3, 2, 9]	[3, 5, 9]	[3, 5, 9]
4	[4, 9, 5, 1, 9]	[5, 9, 4]	[5, 9, 4]
5	[9, 8, 1, 1, 2]	[1, 8, 9]	[1, 8, 9]

The model therefore handled an output sequence shorter than the input sequence without loss of sequence-level accuracy on the held-out test set.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>